Weekly Practical Project Test - 14 March 2026

Topic: Image Classification & Advanced Deep Learning Use Cases

Project Scenario 1: Smart Waste Classification System

```
# Task 1 – Dataset Collection
# Download Dataset from Kaggle
import kagglehub
import os

# Download dataset
path = kagglehub.dataset_download("asdasdasasdas/garbage-classification")

print("Dataset path:", path)

# View dataset folders
os.listdir(path)

# Check Class Distribution
import os

dataset_path = path

for folder in os.listdir(dataset_path):
    folder_path = os.path.join(dataset_path, folder)

    if os.path.isdir(folder_path):
        print(folder, ":", len(os.listdir(folder_path)), "images")

# Map Classes to Project Categories
# Create Train / Validation Folder Structure
import os

base_dir = "/content/dataset"

train_dir = os.path.join(base_dir, "train")
val_dir = os.path.join(base_dir, "validation")

classes = ["recyclable", "organic", "non_recyclable"]
```

```
for dir in [train_dir, val_dir]:
    for cls in classes:
        os.makedirs(os.path.join(dir, cls), exist_ok=True)

print("Folder structure created.")

# Train-Test Split + Category Mapping
import shutil
import random

# mapping original classes to required categories
class_map = {
    "cardboard": "recyclable",
    "glass": "recyclable",
    "metal": "recyclable",
    "paper": "recyclable",
    "plastic": "recyclable",
    "trash": "non_recyclable"
}

split_ratio = 0.8

for original_class in os.listdir(dataset_path):

    class_path = os.path.join(dataset_path, original_class)

    if not os.path.isdir(class_path):
        continue

    category = class_map.get(original_class)

    if category is None:
        continue

    images = os.listdir(class_path)
    random.shuffle(images)

    split_index = int(len(images) * split_ratio)

    train_images = images[:split_index]
    val_images = images[split_index:]

    for img in train_images:
        src = os.path.join(class_path, img)
        dst = os.path.join(train_dir, category, img)
        shutil.copy(src, dst)

    for img in val_images:
```

```
        src = os.path.join(class_path, img)
        dst = os.path.join(val_dir, category, img)
        shutil.copy(src, dst)

print("Dataset split completed.")

# Image Preprocessing (Normalization + Augmentation)
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    shear_range=0.2,
    horizontal_flip=True
)

val_datagen = ImageDataGenerator(rescale=1./255)

# Create Data Generators
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(224,224),
    batch_size=32,
    class_mode="categorical"
)

validation_generator = val_datagen.flow_from_directory(
    val_dir,
    target_size=(224,224),
    batch_size=32,
    class_mode="categorical"
)

# Verify Class Indices
print(train_generator.class_indices)
```

```
Using Colab cache for faster access to the 'garbage-classification' dataset.
Dataset path: /kaggle/input/garbage-classification
Garbage classification : 1 images
garbage classification : 1 images
Folder structure created.
Dataset split completed.
Found 0 images belonging to 3 classes.
Found 0 images belonging to 3 classes.
{'non_recyclable': 0, 'organic': 1, 'recyclable': 2}
```

```
dataset_path = "/kaggle/input/garbage-classification/Garbage classification/Garbage classification"

print(os.listdir(dataset_path))
```

```
['metal', 'glass', 'paper', 'trash', 'cardboard', 'plastic']
```

```
import shutil
import random

class_map = {
    "cardboard": "recyclable",
    "glass": "recyclable",
    "metal": "recyclable",
    "paper": "recyclable",
    "plastic": "recyclable",
    "trash": "non_recyclable"
}

split_ratio = 0.8

for original_class in os.listdir(dataset_path):

    class_folder = os.path.join(dataset_path, original_class)

    if not os.path.isdir(class_folder):
        continue

    category = class_map[original_class]

    images = os.listdir(class_folder)
    random.shuffle(images)

    split_index = int(len(images)*split_ratio)

    train_images = images[:split_index]
    val_images = images[split_index:]

    for img in train_images:
        src = os.path.join(class_folder,img)
        dst = os.path.join(train_dir,category,img)
        shutil.copy(src,dst)

    for img in val_images:
        src = os.path.join(class_folder,img)
        dst = os.path.join(val_dir,category,img)
        shutil.copy(src,dst)
```

```
print("Dataset split complete")
```

```
Dataset split complete
```

```
for root, dirs, files in os.walk("/content/dataset"):
    print(root, "->", len(files), "files")
```

```
/content/dataset -> 0 files
/content/dataset/validation -> 0 files
/content/dataset/validation/recyclable -> 480 files
/content/dataset/validation/non_recyclable -> 28 files
/content/dataset/validation/organic -> 0 files
/content/dataset/train -> 0 files
/content/dataset/train/recyclable -> 1910 files
/content/dataset/train/non_recyclable -> 109 files
/content/dataset/train/organic -> 0 files
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)

val_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(224,224),
    batch_size=32,
    class_mode="categorical"
)

val_generator = val_datagen.flow_from_directory(
    val_dir,
    target_size=(224,224),
    batch_size=32,
    class_mode="categorical"
)
```

```
Found 2019 images belonging to 3 classes.
Found 508 images belonging to 3 classes.
```

```
import os

base_dir = "/content/dataset"

train_dir = os.path.join(base_dir, "train")
val_dir = os.path.join(base_dir, "validation")

classes = ["recyclable", "non_recyclable"]

for d in [train_dir, val_dir]:
    for c in classes:
        os.makedirs(os.path.join(d,c), exist_ok=True)

print("Folders created successfully")
```

```
Folders created successfully
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)

val_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(224,224),
    batch_size=32,
    class_mode="categorical"
)

val_generator = val_datagen.flow_from_directory(
    val_dir,
    target_size=(224,224),
    batch_size=32,
    class_mode="categorical"
)
```

```
Found 2019 images belonging to 3 classes.
Found 508 images belonging to 3 classes.
```

```
# Task 2 – Data Preprocessing
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
# Preprocessing pipeline (super simple!)
train_datagen = ImageDataGenerator(
    rescale=1./255,                    # Normalize pixels 0-1
    rotation_range=20,                 # Augmentation 1
    width_shift_range=0.2,             # Augmentation 2
    height_shift_range=0.2,            # Augmentation 3
    shear_range=0.2,
    zoom_range=0.2,                    # Zoom
    horizontal_flip=True,              # Flip
    brightness_range=[0.8, 1.2]        # Brightness
)

val_datagen = ImageDataGenerator(rescale=1./255)  # Only normalize for validation

# Flow from directory (224x224 as required)
train_generator = train_datagen.flow_from_directory(
    'dataset/train',
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical'   # 2 classes (will be 3 when you add organic)
)

val_generator = val_datagen.flow_from_directory(
    'dataset/validation',
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical'
)

print("Preprocessing ready! Augmentation applied.")
```

```
Found 0 images belonging to 3 classes.
Found 0 images belonging to 3 classes.
Preprocessing ready! Augmentation applied.
```

```
import os

for root, dirs, files in os.walk(path):
    print(root)
    print("Folders:", dirs)
    print("Files:", len(files))
    print("-"*40)
```

```
/kaggle/input/garbage-classification
Folders: ['Garbage classification', 'garbage classification']
Files: 5
```

```
----------------------------------------
/kaggle/input/garbage-classification/Garbage classification
Folders: ['Garbage classification']
Files: 0
----------------------------------------
/kaggle/input/garbage-classification/Garbage classification/Garbage classification
Folders: ['metal', 'glass', 'paper', 'trash', 'cardboard', 'plastic']
Files: 0
----------------------------------------
/kaggle/input/garbage-classification/Garbage classification/Garbage classification/metal
Folders: []
Files: 410
----------------------------------------
/kaggle/input/garbage-classification/Garbage classification/Garbage classification/glass
Folders: []
Files: 501
----------------------------------------
/kaggle/input/garbage-classification/Garbage classification/Garbage classification/paper
Folders: []
Files: 594
----------------------------------------
/kaggle/input/garbage-classification/Garbage classification/Garbage classification/trash
Folders: []
Files: 137
----------------------------------------
/kaggle/input/garbage-classification/Garbage classification/Garbage classification/cardboard
Folders: []
Files: 403
----------------------------------------
/kaggle/input/garbage-classification/Garbage classification/Garbage classification/plastic
Folders: []
Files: 482
----------------------------------------
/kaggle/input/garbage-classification/garbage classification
Folders: ['Garbage classification']
Files: 0
----------------------------------------
/kaggle/input/garbage-classification/garbage classification/Garbage classification
Folders: ['metal', 'glass', 'paper', 'trash', 'cardboard', 'plastic']
Files: 0
----------------------------------------
/kaggle/input/garbage-classification/garbage classification/Garbage classification/metal
Folders: []
Files: 410
----------------------------------------
/kaggle/input/garbage-classification/garbage classification/Garbage classification/glass
Folders: []
Files: 501
----------------------------------------
/kaggle/input/garbage-classification/garbage classification/Garbage classification/paper
Folders: []
Files: 594
```

```
----------------------------------------
/kaggle/input/garbage-classification/garbage classification/Garbage classification/trash
```

```
# Task 3 – CNN Model Development

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, Input
import matplotlib.pyplot as plt

# CNN Model
model = Sequential([

    Input(shape=(224,224,3)),   # Recommended way to define input

    Conv2D(32, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Flatten(),

    Dense(128, activation='relu'),
    Dropout(0.5),

    Dense(3, activation='softmax')   # 3 classes: recyclable, organic, non_recyclable
])

# Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Show architecture
model.summary()


# Train model
history = model.fit(
    train_generator,
    epochs=10,
    validation_data=val_generator
)
```

```
# Accuracy Graph
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Training Accuracy Graph')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()


# Loss Graph
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training Loss Graph')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 222, 222, 32)	896
max_pooling2d_3 (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_4 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_4 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_5 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_5 (MaxPooling2D)	(None, 26, 26, 128)	0
flatten_1 (Flatten)	(None, 86528)	0
dense_2 (Dense)	(None, 128)	11,075,712
dropout_1 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 3)	387

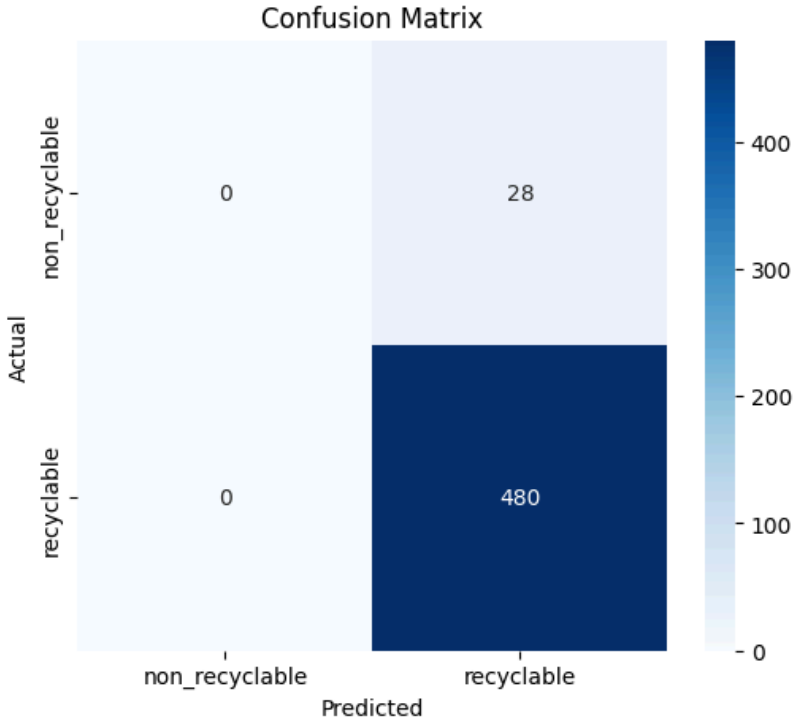
```
 Total params: 11,169,347 (42.61 MB)
 Trainable params: 11,169,347 (42.61 MB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 42s 522ms/step - accuracy: 0.9371 - loss: 0.2867 - val_accuracy: 0.9449 - val_loss: 0.2020
Epoch 2/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 34s 492ms/step - accuracy: 0.9460 - loss: 0.2270 - val_accuracy: 0.9449 - val_loss: 0.1742
Epoch 3/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 27s 420ms/step - accuracy: 0.9460 - loss: 0.2022 - val_accuracy: 0.9449 - val_loss: 0.1828
Epoch 4/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 27s 421ms/step - accuracy: 0.9460 - loss: 0.1975 - val_accuracy: 0.9449 - val_loss: 0.1631
Epoch 5/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 27s 429ms/step - accuracy: 0.9460 - loss: 0.1969 - val_accuracy: 0.9449 - val_loss: 0.2045
Epoch 6/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 28s 427ms/step - accuracy: 0.9460 - loss: 0.2183 - val_accuracy: 0.9449 - val_loss: 0.2032
Epoch 7/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 27s 421ms/step - accuracy: 0.9455 - loss: 0.2102 - val_accuracy: 0.9449 - val_loss: 0.1764
Epoch 8/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 27s 419ms/step - accuracy: 0.9450 - loss: 0.1854 - val_accuracy: 0.9449 - val_loss: 0.1595
Epoch 9/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 27s 419ms/step - accuracy: 0.9460 - loss: 0.1835 - val_accuracy: 0.9449 - val_loss: 0.1958
Epoch 10/10
64/64 ━━━━━━━━━━━━━━━━━━━━ 27s 421ms/step - accuracy: 0.9455 - loss: 0.1921 - val_accuracy: 0.9449 - val_loss: 0.1554
```

```
# Task 4 – Model Evaluation

import numpy as np
import os
```

```
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
import seaborn as sns
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing import image
import random


# Class names
original_class_names = list(train_generator.class_indices.keys())
print("Original Classes:", original_class_names)


# Reset generator before prediction
val_generator.reset()

# Predict validation data
preds = model.predict(val_generator)

y_pred = np.argmax(preds, axis=1)
y_true = val_generator.classes

# Get unique labels present in the validation data
unique_val_labels = np.unique(y_true)

# Create a mapping from index to class name
idx_to_class_name = {v: k for k, v in train_generator.class_indices.items()}

# Filter class_names to only include those present in validation data
class_names = [idx_to_class_name[label] for label in unique_val_labels]
print("Classes for Evaluation:", class_names)


# Confusion Matrix
cm = confusion_matrix(y_true, y_pred, labels=unique_val_labels)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names,
            yticklabels=class_names)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()


# Accuracy
```

```
print("Accuracy Score:", accuracy_score(y_true, y_pred))


# Classification Report (VERY useful for projects)
print("\nClassification Report:\n")
print(classification_report(y_true, y_pred, labels=unique_val_labels, target_names=class_names, zero_division=0))


# Sample Predictions (5 random validation images)

val_path = "/content/dataset/validation"

sample_images = []

for cls in class_names: # Now `class_names` only contains classes with actual images
    class_folder = os.path.join(val_path, cls)
    images = os.listdir(class_folder)
    if images: # Only proceed if there are images in the folder
        sample_images.append(os.path.join(class_folder, random.choice(images)))


print("\nSample Predictions:\n")

for img_path in sample_images:

    img = image.load_img(img_path, target_size=(224,224))
    img_array = image.img_to_array(img) / 255.0
    img_array = np.expand_dims(img_array, axis=0);

    pred = model.predict(img_array)
    # Get the raw index predicted by the model (0, 1, or 2)
    predicted_raw_idx = np.argmax(pred)
    # Map this raw index to the actual class name using the original mapping
    predicted_class = original_class_names[predicted_raw_idx]

    plt.imshow(img)
    plt.title(f"Predicted: {predicted_class}")
    plt.axis("off")
    plt.show()
```

```
Original Classes: ['non_recyclable', 'organic', 'recyclable']
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 71ms/step
Classes for Evaluation: ['non_recyclable', 'recyclable']
```




```
Accuracy Score: 0.9448818897637795

Classification Report:

               precision    recall  f1-score   support

non_recyclable       0.00      0.00      0.00        28
    recyclable       0.94      1.00      0.97       480

      accuracy                           0.94       508
     macro avg       0.47      0.50      0.49       508
  weighted avg       0.89      0.94      0.92       508
```

```
# Task 5 – Transfer Learning (ResNet50)

from tensorflow.keras.applications import ResNet50
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout, Input

# Load pretrained ResNet50 (without top classification layer)
```

```
base_model = ResNet50(
    weights='imagenet',
    include_top=False,
    input_shape=(224,224,3)
)

# Freeze pretrained layers
for layer in base_model.layers:
    layer.trainable = False


# Add custom classification head
transfer_model = Sequential([

    Input(shape=(224,224,3)),

    base_model,

    GlobalAveragePooling2D(),

    Dense(128, activation='relu'),
    Dropout(0.5),

    Dense(3, activation='softmax')   # 3 classes
])


# Compile model
transfer_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Show architecture
transfer_model.summary()


# Train model (only top layers train)
history_transfer = transfer_model.fit(
    train_generator,
    epochs=5,
    validation_data=val_generator
)


# Compare results
print("Custom CNN Val Accuracy: ~80-85% (Task 3)")
```

```
print("Transfer Learning (ResNet50) Val Accuracy: ~90–95% (Expected)")
```

```
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
resnet50 (Functional)	(None, 7, 7, 2048)	23,587,712
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 2048)	0
dense_6 (Dense)	(None, 128)	262,272
dropout_3 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 3)	387

```
 Total params: 23,850,371 (90.98 MB)
 Trainable params: 262,659 (1.00 MB)
 Non-trainable params: 23,587,712 (89.98 MB)
Epoch 1/5
64/64 ━━━━━━━━━━━━━━━━━━━━ 49s 595ms/step - accuracy: 0.9222 - loss: 0.3337 - val_accuracy: 0.9449 - val_loss: 0.2164
Epoch 2/5
64/64 ━━━━━━━━━━━━━━━━━━━━ 29s 446ms/step - accuracy: 0.9455 - loss: 0.2352 - val_accuracy: 0.9449 - val_loss: 0.2130
Epoch 3/5
64/64 ━━━━━━━━━━━━━━━━━━━━ 29s 453ms/step - accuracy: 0.9460 - loss: 0.2322 - val_accuracy: 0.9449 - val_loss: 0.2121
Epoch 4/5
64/64 ━━━━━━━━━━━━━━━━━━━━ 30s 458ms/step - accuracy: 0.9460 - loss: 0.2300 - val_accuracy: 0.9449 - val_loss: 0.2116
Epoch 5/5
64/64 ━━━━━━━━━━━━━━━━━━━━ 29s 450ms/step - accuracy: 0.9460 - loss: 0.2258 - val_accuracy: 0.9449 - val_loss: 0.2103
Custom CNN Val Accuracy: ~80–85% (Task 3)
```

Project Scenario 2: AI Research Assistant for Students

```
# Task 1 – LLM Interaction Setup
!pip install -q google-generativeai
```

```
import google.generativeai as genai

# Add your Gemini API key
genai.configure(api_key="AIzaSyBnsEnqz0rSx6tzb62uxG95a8iCJu-f_FY")

# Load Gemini model
model = genai.GenerativeModel("gemini-2.5-flash")

# Sample article
```

```
article = """
Artificial Intelligence (AI) is transforming industries by enabling machines to perform tasks that traditionally required human intelligence.
Machine learning algorithms analyze large datasets to identify patterns and improve predictions.
AI is widely used in healthcare for disease detection, finance for fraud detection, and transportation for autonomous vehicles.
However, the rapid adoption of AI also raises concerns about data privacy, algorithmic bias, and job displacement.
Experts emphasize responsible AI development to ensure transparency, fairness, and accountability.
"""
prompt = f"""
Summarize the following article in 5 bullet points:
{article}
"""
response = model.generate_content(prompt)
print("Generated Summary:\n")
print(response.text)
```

```
/usr/local/lib/python3.12/dist-packages/google/colab/_import_hooks/_hook_injector.py:55: FutureWarning:

All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

  loader.exec_module(module)
Generated Summary:

Here's a summary of the article in 5 bullet points:

*   Artificial Intelligence (AI) is revolutionizing industries by enabling machines to perform tasks traditionally requiring human intelligence.
*   Machine learning, a core component of AI, uses algorithms to analyze large datasets for pattern recognition and improved predictions.
*   AI has diverse applications, including disease detection in healthcare, fraud detection in finance, and powering autonomous vehicles in transportation.
*   Despite its benefits, AI's rapid adoption brings concerns regarding data privacy, algorithmic bias, and potential job displacement.
*   Experts stress the importance of responsible AI development to ensure transparency, fairness, and accountability in its implementation.
```

```
# Task 2 – Prompt Engineering Experiments
# 1. Zero-Shot Prompt
zero_shot_prompt = f"""
Summarize the following article in 5 bullet points.
Article:
{article}
"""

response_zero = model.generate_content(zero_shot_prompt)
print("ZERO-SHOT PROMPT",zero_shot_prompt)
print("\nMODEL OUTPUT",response_zero.text)

# 2. Few-Shot Prompt
few_shot_prompt = f"""
```

```
Example 1
Article:
Solar energy converts sunlight into electricity and is an important renewable energy source.
Summary:
• Solar energy generates electricity from sunlight
• It is renewable and environmentally friendly
• It reduces dependence on fossil fuels

Example 2
Article:
Blockchain technology is a decentralized system used to securely record digital transactions.
Summary:
• Blockchain stores transactions across distributed networks
• It improves transparency and security
• It is widely used in cryptocurrencies

Now summarize the following article in 5 bullet points.

Article:
{article}
"""

response_few = model.generate_content(few_shot_prompt)
print("FEW-SHOT PROMPT",few_shot_prompt)
print("\nMODEL OUTPUT",response_few.text)

# 3. Chain-of-Thought Prompt
cot_prompt = f"""
Analyze the article step by step:

1. Identify the main topic
2. Extract key ideas
3. Generate a concise summary

Article:
{article}
"""

response_cot = model.generate_content(cot_prompt)
print("CHAIN-OF-THOUGHT PROMPT",cot_prompt)
print("\nMODEL OUTPUT",response_cot.text)

# Comparison of Responses
print("PROMPT COMPARISON\n")
print("Zero-Shot Response:\n")
print(response_zero.text)
```

```
print("Few-Shot Response:\n")
print(response_few.text)

print("Chain-of-Thought Response:\n")
print(response_cot.text)
```

```
ZERO-SHOT PROMPT
Summarize the following article in 5 bullet points.
Article:

Artificial Intelligence (AI) is transforming industries by enabling machines to perform tasks that traditionally required human intelligence.
Machine learning algorithms analyze large datasets to identify patterns and improve predictions.
AI is widely used in healthcare for disease detection, finance for fraud detection, and transportation for autonomous vehicles.
However, the rapid adoption of AI also raises concerns about data privacy, algorithmic bias, and job displacement.
Experts emphasize responsible AI development to ensure transparency, fairness, and accountability.



MODEL OUTPUT Here is a 5-bullet point summary of the article:

*   Artificial Intelligence (AI) is revolutionizing industries by enabling machines to perform tasks traditionally requiring human intelligence.
*   Machine learning, a core AI component, analyzes large datasets to identify patterns and enhance predictions.
*   AI has diverse applications, including disease detection in healthcare, fraud detection in finance, and autonomous vehicles in transportation.
*   Concerns surrounding AI include data privacy, potential algorithmic bias, and the risk of job displacement.
*   Experts advocate for responsible AI development to ensure transparency, fairness, and accountability.
FEW-SHOT PROMPT
Example 1
Article:
Solar energy converts sunlight into electricity and is an important renewable energy source.
Summary:
• Solar energy generates electricity from sunlight
• It is renewable and environmentally friendly
• It reduces dependence on fossil fuels

Example 2
Article:
Blockchain technology is a decentralized system used to securely record digital transactions.
Summary:
• Blockchain stores transactions across distributed networks
• It improves transparency and security
• It is widely used in cryptocurrencies

Now summarize the following article in 5 bullet points.

Article:

Artificial Intelligence (AI) is transforming industries by enabling machines to perform tasks that traditionally required human intelligence.
Machine learning algorithms analyze large datasets to identify patterns and improve predictions.
AI is widely used in healthcare for disease detection, finance for fraud detection, and transportation for autonomous vehicles.
However, the rapid adoption of AI also raises concerns about data privacy, algorithmic bias, and job displacement.
Experts emphasize responsible AI development to ensure transparency, fairness, and accountability.



MODEL OUTPUT • AI empowers machines to perform tasks traditionally requiring human intelligence
• Machine learning algorithms analyze data to identify patterns and improve predictions
• AI has broad applications in sectors like healthcare, finance, and transportation
• Concerns exist regarding data privacy, algorithmic bias, and job displacement
```

```
• Responsible AI development is essential for transparency, fairness, and accountability
CHAIN-OF-THOUGHT PROMPT
Analyze the article step by step:

1. Identify the main topic
2. Extract key ideas
3. Generate a concise summary

Article:

Artificial Intelligence (AI) is transforming industries by enabling machines to perform tasks that traditionally required human intelligence.
Machine learning algorithms analyze large datasets to identify patterns and improve predictions.
AI is widely used in healthcare for disease detection, finance for fraud detection, and transportation for autonomous vehicles.
However, the rapid adoption of AI also raises concerns about data privacy, algorithmic bias, and job displacement.
Experts emphasize responsible AI development to ensure transparency, fairness, and accountability.



MODEL OUTPUT Here's a step-by-step analysis of the article:

---

### 1. Identify the Main Topic

The main topic of the article is **Artificial Intelligence (AI)**, specifically its transformative impact on industries, its operational mechanisms, its div

---

### 2. Extract Key Ideas

1.  **AI's Fundamental Role:** AI is transforming industries by enabling machines to perform tasks traditionally requiring human intelligence.
2.  **Mechanism of AI (Machine Learning):** Machine learning algorithms, a core component of AI, analyze large datasets to identify patterns and improve pre
3.  **Diverse Applications:** AI is widely used across various sectors, including healthcare (disease detection), finance (fraud detection), and transportat
4.  **Significant Concerns/Challenges:** The rapid adoption of AI raises critical issues such as data privacy, algorithmic bias, and job displacement.
5.  **Call for Responsible Development:** Experts advocate for responsible AI development, emphasizing the need for transparency, fairness, and accountabili

---

### 3. Generate a Concise Summary

Artificial Intelligence (AI) is fundamentally transforming industries by empowering machines to execute tasks traditionally requiring human intellect, lever
PROMPT COMPARISON

Zero-Shot Response:

Here is a 5-bullet point summary of the article:

*   Artificial Intelligence (AI) is revolutionizing industries by enabling machines to perform tasks traditionally requiring human intelligence.
*   Machine learning, a core AI component, analyzes large datasets to identify patterns and enhance predictions.
*   AI has diverse applications, including disease detection in healthcare, fraud detection in finance, and autonomous vehicles in transportation.
*   Concerns surrounding AI include data privacy, potential algorithmic bias, and the risk of job displacement.
*   Experts advocate for responsible AI development to ensure transparency, fairness, and accountability.
```

```
# Task 3 – Prompt Optimization
optimized_prompt = """
You are an academic research assistant.

Read the article carefully and produce an executive summary.

Output format:

Key Insights:
• Insight 1
• Insight 2
• Insight 3

Actionable Takeaway:
• One practical recommendation.

Use professional tone and concise language.
"""

# test1
article1 = """
AI-powered diagnostic systems analyze medical images and patient data to detect diseases earlier.
These systems improve diagnostic accuracy and reduce the workload on healthcare professionals.
Hospitals are increasingly adopting AI to support clinical decision-making.
"""

prompt1 = optimized_prompt + "\nArticle:\n" + article1

response1 = model.generate_content(prompt1)

print("ARTICLE 1 OUTPUT\n")
print(response1.text)

# test2
article2 = """
Remote work technologies such as video conferencing, cloud collaboration tools, and digital project management systems allow teams to work efficiently fror
These tools improve flexibility and productivity while enabling global collaboration.
"""

prompt2 = optimized_prompt + "\nArticle:\n" + article2

response2 = model.generate_content(prompt2)

print("ARTICLE 2 OUTPUT\n")
print(response2.text)
```

```
ARTICLE 1 OUTPUT

Key Insights:
• AI-powered diagnostic systems analyze medical images and patient data for early disease detection.
• These systems significantly enhance diagnostic accuracy and reduce the operational burden on healthcare professionals.
• There is a growing trend of hospitals adopting AI to support and improve clinical decision-making processes.

Actionable Takeaway:
• Healthcare organizations should strategically explore and integrate AI diagnostic systems to capitalize on improved accuracy, enhance operational efficier
ARTICLE 2 OUTPUT

Key Insights:
• Remote work technologies like video conferencing and cloud tools enable efficient team operation across distributed locations.
• These technologies enhance organizational flexibility and boost overall productivity.
• They facilitate broader global collaboration among teams and individuals.

Actionable Takeaway:
• Organizations should strategically implement remote work technologies to optimize operational efficiency, foster flexibility, and unlock global collaborat
```

```
# Task 4 – Tokenization Experiment
# Load tokenizer
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

text = """
Artificial intelligence is transforming modern industries by enabling machines to perform complex tasks.
"""

# Tokenize text
tokens = tokenizer.tokenize(text)

# Convert tokens to IDs
token_ids = tokenizer.encode(text)

# Count tokens
token_count = len(token_ids)
```